$x_{1:i-1}, \mathrm{e}(c_i, r_i), x_{i:n}$; we proceed analogously for texts with multiple API calls. Doing this for all $\mathbf{x} \in \mathcal{C}$ results in the new dataset $\mathcal{C}^*$ augmented with API calls. We use this new dataset to finetune $M$, using a standard language modeling objective. Crucially, apart from inserted API calls the augmented dataset $\mathcal{C}^*$ contains the exact same texts as $\mathcal{C}$, the original dataset. As a consequence, finetuning $M$ on $\mathcal{C}^*$ exposes it to the same content as finetuning on $\mathcal{C}$. Moreover, as API calls are inserted in exactly those positions and with exactly those inputs that help $M$ predict future tokens, finetuning on $\mathcal{C}^*$ enables the language model to decide when and how to use which tool, based purely on its own feedback.

**Inference** When generating text with $M$ after finetuning with our approach, we perform regular decoding until $M$ produces the "$\rightarrow$" token, indicating that it next expects the response for an API call. At this point, we interrupt the decoding process, call the appropriate API to get a response, and continue the decoding process after inserting both the response and the `</API>` token.

## 3 Tools

We explore a variety of tools to address different shortcomings of regular LMs. The only constraints we impose on these tools is that (i) both their inputs and outputs can be represented as text sequences, and (ii) we can obtain a few demonstrations of their intended use. Concretely, we explore the following five tools: a question answering system, a Wikipedia search engine, a calculator, a calendar, and a machine translation system. Some examples of potential calls and return strings for the APIs associated with each of these tools are shown in Table 1. We briefly discuss all tools below; further details can be found in Appendix A.

**Question Answering** Our first tool is a question answering system based on another LM that can answer simple factoid questions. Specifically, we use *Atlas* (Izacard et al., 2022), a retrieval-augmented LM finetuned on Natural Questions (Kwiatkowski et al., 2019).

**Calculator** As a second tool, we use a calculator that can perform simple numeric calculations; we only support the four basic arithmetic operations. Results are always rounded to two decimal places.

**Wikipedia Search** Our third tool is a search engine that, given a search term, returns short text snippets from Wikipedia. Compared to our question answering tool, this search enables a model to get more comprehensive information on a subject, but requires it to extract the relevant parts by itself. As our search engine, we use a BM25 retriever (Robertson et al., 1995; Baeza-Yates et al., 1999) that indexes the Wikipedia dump from KILT (Petroni et al., 2021).

**Machine Translation System** Our fourth tool is a machine translation system based on a LM that can translate a phrase from any language into English. More concretely, we use the 600M parameter NLLB (Costa-jussà et al., 2022) as our multilingual machine translation model that works for 200 languages (including low-resource ones). The source language is automatically detected using the *fastText* classifier (Joulin et al., 2016), while the target language is always set to English.

**Calendar** Our final tool is a calendar API that, when queried, returns the current date without taking any input. This provides temporal context for predictions that require some awareness of time.

## 4 Experiments

We investigate whether our approach enables a model to use tools without any further supervision and to decide for itself when and how to call which of the available tools. To test this, we select a variety of downstream tasks where we assume at least one of the considered tools to be useful, and evaluate performance in zero-shot settings (Section 4.2). Beyond that, we also ensure that our approach does not hurt the model's core language modeling abilities; we verify this by looking at perplexity on two language modeling datasets (Section 4.3). Finally, we investigate how the ability to learn using tools is affected by model size (Section 4.4).

### 4.1 Experimental Setup

**Dataset Generation** Throughout all of our experiments, we use a subset of CCNet (Wenzek et al., 2020) as our language modeling dataset $\mathcal{C}$ and GPT-J (Wang and Komatsuzaki, 2021) as our language model $M$. To reduce the computational cost of annotating $\mathcal{C}$ with API calls, we define heuristics for some APIs to get a subset of $\mathcal{C}$ for which API calls are more likely to be helpful than for an average text. For example, we only consider texts for the calculator tool if they contain at least three numbers. Details of the heuristics used are given in

| API Name | Example Input | Example Output |
|---|---|---|
| Question Answering | Where was the Knights of Columbus founded? | New Haven, Connecticut |
| Wikipedia Search | Fishing Reel Types | Spin fishing > Spin fishing is distinguished between fly fishing and bait cast fishing by the type of rod and reel used. There are two types of reels used when spin fishing, the open faced reel and the closed faced reel. |
| Calculator | 27 + 4 * 2 | 35 |
| Calendar | $\varepsilon$ | Today is Monday, January 30, 2023. |
| Machine Translation | sûreté nucléaire | nuclear safety |

Table 1: Examples of inputs and outputs for all APIs used.

| API | Number of Examples | | |
|---|---|---|---|
| | $\tau_f = 0.5$ | $\tau_f = 1.0$ | $\tau_f = 2.0$ |
| Question Answering | 51,987 | 18,526 | 5,135 |
| Wikipedia Search | 207,241 | 60,974 | 13,944 |
| Calculator | 3,680 | 994 | 138 |
| Calendar | 61,811 | 20,587 | 3,007 |
| Machine Translation | 3,156 | 1,034 | 229 |

Table 2: Number of examples with API calls in $\mathcal{C}^*$ for different values of our filtering threshold $\tau_f$.

Appendix A. For obtaining $\mathcal{C}^*$ from $\mathcal{C}$, we perform all steps described in Section 2 and additionally filter out all examples for which all API calls were eliminated in the filtering step.[4] For the weighting function, we use

$$w_t = \frac{\tilde{w}_t}{\sum_{s \in \mathbb{N}} \tilde{w}_s} \text{ with } \tilde{w}_t = \max(0, 1 - 0.2 \cdot t)$$

to make sure that API calls happen close to where the information provided by the API is actually helpful for the model. The thresholds $\tau_s$ and $\tau_f$ are chosen individually for each tool to ensure a sufficiently larger number of examples; see Appendix A for details. Table 2 shows relevant statistics of our final dataset augmented with API calls.

**Model Finetuning** We finetune $M$ on $\mathcal{C}^*$ using a batch size of 128 and a learning rate of $1 \cdot 10^{-5}$ with linear warmup for the first 10% of training. Details of our finetuning procedure are given in Appendix B.

**Baseline Models** Throughout the remainder of this section, we mainly compare the following models:

- **GPT-J**: A regular GPT-J model without any finetuning.
- **GPT-J + CC**: GPT-J finetuned on $\mathcal{C}$, our subset of CCNet *without* any API calls.
- **Toolformer**: GPT-J finetuned on $\mathcal{C}^*$, our subset of CCNet augmented with API calls.
- **Toolformer (disabled)**: The same model as Toolformer, but API calls are disabled during decoding.[5]

For most tasks, we additionally compare to OPT (66B) (Zhang et al., 2022) and GPT-3[6] (175B) (Brown et al., 2020), two models that are about 10 and 25 times larger than our other baseline models, respectively.

### 4.2 Downstream Tasks

We evaluate all models on a variety of downstream tasks. In all cases, we consider a prompted zero-shot setup – i.e., models are instructed to solve each task in natural language, but we do not provide any in-context examples. This is in contrast to prior work on tool use (e.g., Gao et al., 2022; Parisi et al., 2022), where models are provided with dataset-specific examples of how a tool can be used to solve a concrete task. We choose the more challenging zero-shot setup as we are interested in seeing whether Toolformer works in precisely those cases where a user does not specify in advance which tools should be used in which way for solving a specific problem.

We use standard greedy decoding, but with one modification for Toolformer: We let the model start an API call not just when `<API>` is the most likely

[4] While this filtering alters the distribution of training examples, we assume that the remaining examples are close enough to the original distribution so that $M$'s language modeling abilities remain unaffected. This assumption is empirically validated in Section 4.3.

[5] This is achieved by manually setting the probability of the `<API>` token to 0.

[6] We use the original `davinci` variant that is not finetuned on any instructions.

token, but whenever it is one of the $k$ most likely tokens. For $k = 1$, this corresponds to regular greedy decoding; we instead use $k = 10$ to increase the disposition of our model to make use of the APIs that it has access to. At the same time, we only at most one API call per input to make sure the model does not get stuck in a loop where it constantly calls APIs without producing any actual output. The effect of these modifications is explored in Section 5.

#### 4.2.1 LAMA

We evaluate our models on the SQuAD, Google-RE and T-REx subsets of the LAMA benchmark (Petroni et al., 2019). For each of these subsets, the task is to complete a short statement with a missing fact (e.g., a date or a place). As LAMA was originally designed to evaluate *masked* language models (e.g., Devlin et al., 2019), we filter out examples where the mask token is not the final token, so that the remaining examples can be processed in a left-to-right fashion. To account for different tokenizations and added complexity from not informing the model that a single word is required, we use a slightly more lenient evaluation criterion than exact match and simply check whether the correct word is within the first five words predicted by the model. As LAMA is based on statements obtained directly from Wikipedia, we prevent Toolformer from using the Wikipedia Search API to avoid giving it an unfair advantage.

Results for all models can be seen in Table 3. All GPT-J models without tool use achieve similar performance. Crucially, Toolformer clearly outperforms these baseline models, improving upon the best baseline by 11.7, 5.2 and 18.6 points, respectively. It also clearly outperforms OPT (66B) and GPT-3 (175B), despite both models being much larger. This is achieved because the model independently decides to ask the question answering tool for the required information in almost all cases (98.1%); for only very few examples, it uses a different tool (0.7%) or no tool at all (1.2%).

#### 4.2.2 Math Datasets

We test mathematical reasoning abilities on ASDiv (Miao et al., 2020), SVAMP (Patel et al., 2021) and the MAWPS benchmark (Koncel-Kedziorski et al., 2016). We again account for the fact that we test all models in a zero-shot setup by using a more lenient evaluation criterion: As the required output is always a number, we simply check for the first number predicted by the model.[7]

| Model | SQuAD | Google-RE | T-REx |
|---|---|---|---|
| GPT-J | 17.8 | 4.9 | 31.9 |
| GPT-J + CC | 19.2 | 5.6 | 33.2 |
| Toolformer (disabled) | 22.1 | 6.3 | 34.9 |
| Toolformer | **33.8** | **11.5** | **53.5** |
| OPT (66B) | 21.6 | 2.9 | 30.1 |
| GPT-3 (175B) | 26.8 | 7.0 | 39.8 |

Table 3: Results on subsets of LAMA. Toolformer uses the question answering tool for most examples, clearly outperforming all baselines of the same size and achieving results competitive with GPT-3 (175B).

| Model | ASDiv | SVAMP | MAWPS |
|---|---|---|---|
| GPT-J | 7.5 | 5.2 | 9.9 |
| GPT-J + CC | 9.6 | 5.0 | 9.3 |
| Toolformer (disabled) | 14.8 | 6.3 | 15.0 |
| Toolformer | **40.4** | **29.4** | **44.0** |
| OPT (66B) | 6.0 | 4.9 | 7.9 |
| GPT-3 (175B) | 14.0 | 10.0 | 19.8 |

Table 4: Results for various benchmarks requiring mathematical reasoning. Toolformer makes use of the calculator tool for most examples, clearly outperforming even OPT (66B) and GPT-3 (175B).

Table 4 shows results for all benchmarks. While GPT-J and GPT-J + CC perform about the same, Toolformer achieves stronger results even when API calls are disabled. We surmise that this is because the model is finetuned on many examples of API calls and their results, improving its own mathematical capabilities. Nonetheless, allowing the model to make API calls more than doubles performance for all tasks, and also clearly outperforms the much larger OPT and GPT-3 models. This is because across all benchmarks, for 97.9% of all examples the model decides to ask the calculator tool for help.

#### 4.2.3 Question Answering

We look at Web Questions (Berant et al., 2013), Natural Questions (Kwiatkowski et al., 2019) and TriviaQA (Joshi et al., 2017), the three question answering datasets considered by Brown et al. (2020). For evaluation, we check whether the first 20 words predicted by a model contain the correct answer instead of requiring an exact match. For Toolformer, we disable the question answering tool as

[7] An exception to this is if the model's prediction contains an equation (e.g., "The correct answer is 5+3=8"), in which case we consider the first number after the "=" sign to be its prediction.